CnA FRAMEWORK

FRONTEND ENGINE

Technical Architecture & Implementation Plan

Vite integration ■ Modular UI system ■ PHP View Engine ■ React / Vue

adapters

Purpose

Define a production-minded frontend subsystem for CnA Framework that integrates Vite with the PHP backend while

preserving strict modular boundaries and allowing multiple frontend approaches.

Architecture baseline: PHP 8.4+ · strict modular monolith · Composer packages · Vite-managed frontend

assets · MySQL / PostgreSQL / SQLite compatibility retained

Prepared as an engineering blueprint. This document describes the proposed architecture; it does not claim that the subsystem has already

been implemented or production-certified.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Document overview

This plan introduces the CnA Frontend Engine as a first-class subsystem. Vite is the build and development tool,

CnA owns the PHP integration and rendering contracts, and UI libraries such as React or Vue remain optional

adapters.

Recommended architectural decision

Vite is the official frontend asset pipeline; CnA View is the server-rendering layer; CnA UI Modules define

frontend entry-point organization. Keep the default path lightweight with TypeScript/vanilla JavaScript and

CSS. Add React and Vue as optional, separately installable integrations.

What this plan covers

■ Subsystem boundaries, package responsibilities, and request/build flows.

■ Vite development-server integration and production manifest resolution.

■ PHP view helpers, module registration, component conventions, and CLI commands.

■ Optional React/Vue adapters, API integration, security, performance, testing, and deployment.

■ A staged implementation roadmap with acceptance criteria and release gates.

Scope and non-goals

■ In scope: asset pipeline, PHP view integration, frontend module conventions, component scaffolding, optional UI

adapters, and build tooling.

■ Not in the initial scope: forcing a SPA architecture, bundling Node.js into PHP runtime, building a proprietary JavaScript

framework, or automatically converting arbitrary PHP views into React/Vue.

■ SSR and full SPA routing are future capabilities, not prerequisites for the first stable release.

1. Architecture principles

Principle

Design rule

Strict modularity

Frontend integration is a package boundary with explicit contracts; core HTTP and database packages

must not depend on Vite internals.

Backend/frontend separation

PHP owns business logic, authorization, sessions, and server routes. Vite owns frontend compilation and

development serving.

Progressive enhancement

A PHP-rendered page must remain usable without React/Vue. JavaScript enhances only where needed.

Technology neutrality

Vanilla JS/TypeScript is the baseline. React and Vue are optional adapters.

Build-time dependency

Node.js is required to develop/build assets, not to serve prebuilt assets in production.

Explicit entry points

Only registered/configured entry points may be emitted; never accept arbitrary request-controlled asset

paths.

Observable and testable

Manifest resolution, asset tags, dev mode, and module loading have unit and integration tests.

2. High-level architecture

The Frontend Engine sits between the CnA View Engine and Vite. It resolves assets and exposes stable PHP APIs

without leaking Vite-specific behavior into controllers or business services.

Layer

Main responsibilities

CnA Application

Routes, controllers, middleware, authentication, authorization, APIs, configuration.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Layer

Main responsibilities

CnA View Engine

Layouts, templates, server-rendered components, escaping, view context.

CnA Frontend Integration

Asset facade, entry-point registry, dev-server detection, manifest reader, URL generation, preload and

stylesheet collection.

Vite

Development server, HMR, dependency graph, CSS processing, code splitting, production build, manifest

output.

UI adapters

Optional React/Vue bootstrapping, mounting conventions, adapter-specific plugins.

Loads HTML, CSS and JavaScript; calls authenticated CnA endpoints.

Dependency direction: application → view/frontend contracts → frontend implementation. The frontend

package may depend on CnA contracts and view extension points; the core kernel must not depend on React,

Vue, or Vite packages.

3. Package and repository structure

Recommended Composer monorepo additions: cna/view and cna/frontend. Framework adapters should be separate

packages so the base installation remains small.

cna-framework/

■■■ packages/

■ ■■■ contracts/

■ ■■■ core/

■ ■■■ http/

■ ■■■ routing/

■ ■■■ console/

■ ■■■ view/

■ ■ ■■■ src/

■ ■ ■■■ Contracts/

■ ■ ■■■ Engine/

■ ■ ■■■ Renderer/

■ ■ ■■■ Layouts/

■ ■ ■■■ Components/

■ ■ ■■■ Helpers/

■ ■■■ frontend/

■ ■ ■■■ src/

■ ■ ■■■ Contracts/

■ ■ ■■■ AssetManager.php

■ ■ ■■■ ManifestReader.php

■ ■ ■■■ DevServer.php

■ ■ ■■■ AssetResolver.php

■ ■ ■■■ PreloadManager.php

■ ■ ■■■ FrontendServiceProvider.php

■ ■■■ frontend-react/ # optional

■ ■■■ frontend-vue/ # optional

■■■ resources/

■ ■■■ views/

■ ■ ■■■ layouts/

■ ■ ■■■ components/

■ ■ ■■■ pages/

■ ■■■ js/

■ ■■■ css/

■ ■■■ modules/

■■■ public/

■ ■■■ build/

■■■ vite.config.js

■■■ package.json

■■■ composer.json

Package responsibility matrix

Package

Owns

Must not own

cna/view

PHP templates, layouts, rendering, safe output helpers

Vite process lifecycle or React runtime

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Package

Owns

Must not own

cna/frontend

Manifest, asset resolution, dev-server URLs, asset tags,

preload collection

Business logic, auth decisions, UI framework internals

cna/frontend-react

React entry conventions and mount adapter

Core asset resolution implementation

cna/frontend-vue

Vue entry conventions and mount adapter

Core asset resolution implementation

Application

Page routes, controller data, registered entry points

Direct manipulation of generated hashed filenames

4. Runtime and build workflows

Development request flow

■ Developer starts the CnA PHP application server and Vite development server as separate processes.

■ A request reaches the PHP router, middleware, controller, and View Engine.

■ The view asks the Frontend Engine to render a registered entry point.

■ The Frontend Engine detects development mode and emits the Vite client script plus the configured source entry URL.

■ The browser loads source modules from Vite; HMR updates the frontend without restarting PHP.

Production build and request flow

■ CI or the deployment pipeline installs locked frontend dependencies and runs the Vite production build.

■ Vite writes fingerprinted assets and a manifest into the configured public build directory.

■ The PHP deployment receives the application code and generated build artifacts.

■ At runtime, the Frontend Engine reads/caches the manifest, resolves the registered entry, emits required CSS and JS

tags, and the browser fetches static assets from the web server/CDN.

■ Node.js and the Vite development server are not required for ordinary production PHP requests.

Core sequence

|

v

Frontend AssetManager

|

Vite dev server URL

|

|

|

hashed CSS / JS / preload tags

|

Important operational rule

PHP and Vite are separate processes. The PHP application must not shell out to start Vite on every request.

Starting/stopping the development server belongs to the CLI or local development tooling.

5. Configuration and Vite setup

Keep frontend configuration explicit and environment-aware. A suggested application configuration is:

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

'enabled' => true,

'mode' => env('APP_ENV') === 'production' ? 'production' : 'development',

'dev_server' => [

'url' => env('VITE_DEV_SERVER_URL', 'http://127.0.0.1:5173'),

'allowed_origins' => ['http://localhost:8000'],

],

'build' => [

'directory' => public_path('build'),

'manifest' => public_path('build/.vite/manifest.json'),

'public_path' => '/build/',

],

'entry_points' => [

'app' => 'resources/js/app.ts',

'styles' => 'resources/css/app.css',

],

Illustrative Vite configuration

host: '127.0.0.1',

port: 5173,

strictPort: true,

},

outDir: 'public/build',

emptyOutDir: true,

manifest: true,

input: [

'resources/js/app.ts',

'resources/css/app.css',

],

},

},

This is a baseline only. The final Vite config should be generated by the CnA installer and should match the supported Vite major version and

official manifest format. Do not hard-code a Vite major version in the PHP package.

6. PHP asset integration API

The public API should be small and stable. Keep implementation classes injectable so the facade remains optional.

// Render the configured application entry point:

// Or render a registered module entry:

Recommended internal contracts

Contract / service

Responsibility

EntryPointRegistry

Maps stable logical names such as 'dashboard' to configured source entry paths.

DevServerResolver

Determines whether dev mode is enabled and validates the configured dev-server URL.

ManifestReaderInterface

Loads, validates, and caches the production manifest.

AssetResolverInterface

Resolves entry files, imports, CSS, and static assets into safe public URLs.

AssetTagRenderer

Renders escaped script, stylesheet, and module-preload tags.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Contract / service

Responsibility

PreloadManager

Collects dependency preloads and deduplicates tags per response.

Manifest resolution algorithm

■ Resolve the logical entry point through the explicit registry; reject unknown entries.

■ In development mode, validate scheme, host, port, and trusted origin; emit the Vite client and source entry.

■ In production, load the manifest from a fixed configured path and validate its structure.

■ Find the manifest record for the registered source entry; fail with a clear diagnostic if missing.

■ Recursively traverse static imports; collect CSS and module-preload candidates while preserving dependency order.

■ Deduplicate CSS and preload URLs across the current response.

■ Escape all emitted attribute values and ensure resolved files remain within the configured build root.

■ Cache parsed manifest data using a deployment-safe cache strategy; invalidate on manifest version/mtime change.

Illustrative manifest-aware output

The filenames above are examples only. The actual output must always be derived from the generated

manifest, never guessed or manually maintained.

7. CnA UI Module architecture

A UI module is a frontend feature boundary, not necessarily a Composer package. It groups one or more entry points,

styles, and browser-side components for a feature such as dashboard, users, analytics, or settings.

■■■ modules/

■ ■■■ dashboard/

■ ■ ■■■ index.ts

■ ■ ■■■ dashboard.css

■ ■ ■■■ components/

■ ■■■ users/

■ ■ ■■■ index.ts

■ ■ ■■■ components/

■ ■■■ analytics/

■ ■■■ index.ts

■ ■■■ components/

■■■ js/

■ ■■■ bootstrap.ts

■■■ css/

■■■ app.css

Module conventions

■ Each module has a stable identifier and one or more explicit Vite entry points.

■ Modules may declare frontend dependencies and styles, but must not contain backend authorization logic.

■ The PHP route/controller decides which module is rendered; the browser only receives the assets for that page.

■ Shared code belongs in shared frontend libraries, not copied between modules.

■ Dynamic imports may be used for large optional features; test loading and error behavior.

■ Module registration must be deterministic and should not rely on arbitrary filesystem scanning in production.

Example page integration

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

The entry module reads the DOM mount point and calls the relevant UI adapter. Keep API endpoint configuration

explicit and avoid embedding secrets in frontend code.

8. Optional React and Vue adapters

React and Vue should be optional packages. They provide bootstrapping conventions and Vite plugin setup; they do

not replace the CnA View Engine or the PHP backend.

Capability

Vanilla / TypeScript

React adapter

Vue adapter

Default installation

Yes

No

No

Component model

DOM modules / custom JS

React components

Vue components

Build tool

Vite

Vite + React plugin

Vite + Vue plugin

Best fit

Progressive enhancement, small

widgets

Complex dashboards and highly

interactive screens

Reactive forms and

component-driven interfaces

PHP integration

View helper + DOM mount

View helper + mount root

View helper + mount root

React mount example

Adapter boundaries

■ The adapter owns framework-specific mounting and hydration behavior.

■ The shared Frontend Engine owns manifest resolution, asset URLs, and tag generation.

■ Do not expose React/Vue types in the base frontend package's public PHP contracts.

■ Keep SSR and hydration protocols isolated in future packages with explicit process and failure boundaries.

9. CnA server-rendered UI components

Build a small PHP component convention that works without a JavaScript framework. This makes CnA suitable for

conventional PHP pages and progressive enhancement.

'label' => 'Create user',

'variant' => 'primary',

'size' => 'medium',

]) ?>

Initial component catalogue

■ Form controls: input, select, checkbox, radio, textarea, validation message.

■ Navigation: navbar, sidebar, breadcrumb, tabs, pagination.

■ Feedback: alert, toast, badge, loading indicator, empty state.

■ Overlays: modal, dropdown, popover, confirmation dialog.

■ Data display: table, card, statistic tile, chart container.

■ Utilities: responsive grid, stack, container, spacing and typography primitives.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Component requirements

■ Accessible HTML semantics, keyboard support, focus management, and visible focus styles.

■ Escape output by default; require explicit safe/raw output APIs for trusted HTML.

■ Document required props, optional props, defaults, events, and accessibility behavior.

■ Keep visual tokens/theme variables consistent; do not make a component library a hard dependency of the framework

core.

■ Use CSS variables/design tokens so application themes can override appearance without forking components.

10. CLI and developer experience

Extend the existing CnA CLI with frontend commands. The PHP CLI should orchestrate supported workflows, but

package installation and Vite execution remain Node/npm processes.

Command

Expected behavior

optionally invoke package manager only with explicit user consent.

Launch or guide the developer to launch the configured Vite dev server; report URL and port

conflicts.

Run the configured production build and verify manifest/output existence.

Report mode, configured entry points, Vite URL reachability, manifest status, and build

output.

Generate module directory, entry file, stylesheet, and registration example.

Generate a documented PHP component or frontend component according to an explicit

target option.

Clear CnA's cached manifest/asset metadata, not user source files.

Suggested developer workflow

cd my-app

Command semantics should be finalized during implementation. In particular, do not make a PHP command silently install arbitrary npm

packages or run shell commands without safe argument handling and clear output.

11. Security architecture

The Frontend Engine processes build metadata and emits executable resource references. Treat the manifest,

development-server configuration, and asset paths as security-sensitive inputs.

Threat / area

Required control

Path traversal

Allow only registered entry keys; normalize resolved paths; reject traversal, absolute paths, and symlink

escapes outside the configured build root.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Threat / area

Required control

Manifest tampering

Read from a fixed trusted location; validate JSON schema and expected fields; deploy build artifacts with

controlled ownership and permissions.

Dev server exposure

Bind to loopback by default; allow only explicit trusted origins; never enable public dev server access by

default.

XSS / HTML injection

Escape generated URLs and attributes; PHP view output escapes by default; isolate explicitly trusted raw

HTML.

CSP

Support nonce propagation for inline scripts where needed; prefer external hashed assets and avoid

unnecessary inline executable code.

CSRF / API security

Use CnA CSRF protection for session-authenticated state-changing requests; enforce authorization

server-side.

Secrets

Never place API keys, database credentials, private tokens, or server secrets in frontend bundles or Vite

environment variables exposed to the client.

Dependency risk

Commit lockfiles, audit dependencies, review transitive packages, and pin tested versions for release

builds.

Cache safety

Use content-hashed immutable assets; ensure deployments do not leave stale HTML referencing deleted

assets.

Non-negotiable rule

Frontend code is untrusted from the backend's perspective. Hiding a button in React/Vue is not authorization.

Every protected operation must be authorized by CnA middleware, policy, or application service.

12. Performance and reliability

Concern

Design approach

Code splitting

Use explicit entry points and dynamic imports for large optional features; avoid shipping all admin modules

on every page.

CSS loading

Collect CSS dependencies from manifest imports, preserve dependency ordering, and deduplicate tags.

Preloading

Preload only relevant entry dependencies; avoid excessive preload hints that compete with critical assets.

Manifest caching

Cache parsed manifest per deployment/version; avoid reading and decoding the same file repeatedly on

every request.

Static asset delivery

Serve fingerprinted assets through Nginx/Apache or a CDN with long-lived immutable caching.

Compression

Enable Brotli or gzip at the web server/CDN; avoid compressing already-compressed formats unnecessarily.

Failure behavior

In production, missing manifest/entry should produce a clear logged error and controlled failure, not silently emit broken URLs.

Observability

Log build version, unresolved entry keys, manifest load failures, and dev-server connectivity issues without exposing secrets.

13. Build and deployment architecture

Recommended deployment separates the frontend build stage from the PHP runtime stage.

Developer machine:

PHP runtime + CnA app

Node.js + package manager + Vite dev server

checkout source

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

install PHP dependencies (Composer lock)

install JS dependencies (npm ci / equivalent)

run frontend tests and lint

run Vite production build

validate manifest and generated assets

package PHP application + public/build

web server / PHP-FPM -> CnA PHP application

web server / CDN -> public/build fingerprinted assets

no Vite dev server required

no Node process required for normal asset serving

Deployment checklist

■ Build assets from a committed lockfile using a supported Node.js LTS version.

■ Verify manifest and every referenced file exist before deployment.

■ Deploy HTML/PHP code and its matching asset build atomically or use versioned release directories.

■ Configure static asset cache headers for fingerprinted files.

■ Ensure web server permissions allow reading build files but do not allow public writes.

■ Keep source maps private or publish them only under an intentional policy.

■ Rollback PHP code and asset build as a matched release.

14. Testing and quality strategy

Test layer

Test cases / exit criteria

Unit

Manifest parsing, entry registry, path normalization, URL generation, CSS/import traversal, duplicate

suppression, HTML escaping.

Integration

Development-mode tags, production-mode tags, missing manifest, missing entry, nested imports, CSS

dependencies, multiple modules per response.

Security

Traversal attempts, malicious manifest fields, untrusted dev-server origins, XSS in attributes, symlink escape,

invalid schemes.

End-to-end

A sample CnA page loads its CSS/JS; HMR works in development; production build works without a running

Vite server.

Compatibility

Test supported PHP versions and supported Vite major versions; test vanilla baseline and each optional adapter

independently.

Performance

Benchmark manifest cache hits, large dependency graphs, and rendered asset tag generation.

Release gates

■ No critical or high-severity unresolved security findings in the frontend integration package.

■ All supported build modes pass integration tests.

■ Unknown/unregistered entry points fail safely.

■ Production pages do not require the Vite development server or Node runtime.

■ Documentation includes installation, configuration, development, build, deployment, troubleshooting, and adapter setup.

■ A clean sample application can be installed and built from documented commands.

15. Implementation roadmap

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Phase

Scope

Deliverable / exit criteria

0 — Design freeze

Finalize package contracts, config schema,

supported PHP/Node/Vite versions, error model.

Approved interfaces and architecture decision records.

1 — View

foundation

Templates, layouts, component rendering, escaping,

view tests.

PHP pages render safely without JavaScript.

2 — Vite core

Dev server resolver, manifest reader, asset resolver,

CSS/import traversal, tag renderer.

Vanilla TS/CSS sample works in dev and production.

3 — UI modules +

CLI

Entry registry, module generator, component

scaffolding, frontend status/build commands.

Two independent feature modules load only on their pages.

4 — UI toolkit

Accessible PHP components, design tokens,

responsive layout primitives.

Sample admin UI built from reusable components.

5 — Adapters

Optional React and Vue packages, adapter-specific

Vite setup and tests.

React and Vue examples work without changing core

package.

6 — Advanced

rendering

Evaluate SSR/hydration and SPA integration

separately.

Architecture decision and isolated prototype; no core

coupling.

Suggested first milestone: CnA Frontend Engine v0.1

■ cna/view package with layouts and safe PHP rendering.

■ cna/frontend package with AssetManager, ManifestReader, DevServer, AssetResolver, and service provider.

■ Vite dev mode and production manifest mode.

■ Automatic entry CSS, imported CSS, and JavaScript tag generation.

■ Vanilla TypeScript sample dashboard.

■ CLI commands for frontend status, build verification, and UI module scaffolding.

■ Unit, integration, and security test coverage for the asset pipeline.

16. Architecture decisions to lock before coding

Decision

Recommended default

Vite role

Official build/dev tool, not a PHP runtime dependency.

Default frontend

Vanilla JavaScript + TypeScript + CSS.

UI frameworks

React and Vue are optional adapters, not mandatory dependencies.

Rendering model

PHP View Engine remains the default server-rendered approach.

Module model

Explicitly registered UI module entry points.

Production assets

Vite manifest + fingerprinted files under public/build.

SSR / SPA

Future opt-in capabilities in separate packages or adapters.

CLI

CnA CLI orchestrates and validates; Node package manager remains responsible for JS dependencies.

17. Risks and mitigations

Risk

Mitigation

Vite version changes manifest behavior

Pin and test supported Vite major versions; keep manifest parser behind an interface; run compatibility tests before upgrades.

CnA FRAMEWORK / FRONTEND ENGINE

Technical Architecture Plan | Version 1.0 | 13 November 2026

Risk

Mitigation

Frontend scope expands too early

Ship vanilla baseline first; React/Vue/SSR/SPA remain optional roadmap items.

PHP and frontend build artifacts become

mismatched

Use atomic/versioned deployments and validate manifest assets before release.

UI components become tightly coupled to

one CSS framework

Use design tokens and adapter boundaries; keep styling choices replaceable.

Developers bypass asset registry

Document and enforce stable logical entry names; restrict asset helper inputs.

Frontend authorization is mistaken for

security

Keep all access control in PHP middleware/policies/services and test direct API calls.

18. Final recommendation

Adopt the CnA Frontend Engine as a separate modular subsystem with three clear layers:

■ CnA View: PHP templates, layouts, safe server-rendered components.

■ CnA Frontend: Vite integration, manifest resolution, entry-point registry, asset tags, preload management, and CLI

support.

■ CnA UI Modules / Adapters: feature-oriented frontend modules, with vanilla TypeScript as the baseline and optional

React/Vue integrations.

This design adds modern interactive UI capabilities without compromising the framework's strict modular

architecture or making a JavaScript framework mandatory. It also creates a controlled path toward admin

tooling, SSR, and SPA applications as later, independently validated capabilities.

Official technical references

■ Vite — Backend Integration: https://vite.dev/guide/backend-integration.html

■ Vite — Build Options / manifest: https://vite.dev/config/build-options.html

■ Vite — Features and development server: https://vite.dev/guide/features.html

■ React — createRoot API: https://react.dev/reference/react-dom/client/createRoot

■ Vue — Quick Start: https://vuejs.org/guide/quick-start.html

Document status: Proposed architecture baseline. Implementation details, dependency versions, compatibility matrix, and public APIs must be

validated in code and tests before a stable release.